

Audience: LEWG, LWG, WG21
Document number: D3945R0
Date: 2025-12-12
Project: Comments on D3933R0 (constexpr std::hive)
Reply-to: Matthew Bentley <mattreecebentley@gmail.com>

COMMENTS ON D3933R0 (CONSTEXPR STD::HIVE PROPOSAL)

TLDR; `get_iterator` might be possible in `constexpr`, history of `hive` is incorrect (not entirely author's doing), encourage author to report `block_capacity_hard_limits` issue to LWG, both implementations have UB, there were faults in procedures for pursuing this, NB comments covering large changes warrant breaking out into external discussion, the paper's implementation does not compile with supplied build instructions and cannot test, I would think better of `constexpr` `hive` if a complete + testable implementation is provided, people other than I and the authors test it, and it presents no issues nor architecturally-intrinsic performance problems.

Table of Contents

- I. [Technical comments](#)
- II. [History correction](#)
- III. [Implementation comments](#)
- IV. [Procedures comments](#)

I. Technical comments

1. The exclusion of `get_iterator` from `constexpr` may not be necessary, however the author would need to talk to people more familiar with the territory. We had a similar issue in `get_iterator` at runtime;

see the note under `get_iterator` in the hive paper under design decisions->specific functions->`get_iterator`:

"Note 2: `get_iterator` compares pointers against the start and end memory locations of the active blocks in `*this`. There was some confusion that this would be problematic due to obscure rules in the standard which state that a given platform may allow addresses inside of a given memory block to essentially not be contiguous, at least in terms of the `std::less/std::greater/>/</etc` operators. According to Jens Maurer, these difficulties can be bypassed via hidden channels between the library implementation and the compiler."

The relevant talk discussing this is on the lewg reflector, under the heading "Re: [isocpp-lib-ext] Notes on P0447R20, was Re: Info+paper for hive discussion Tuesday 5th 11am pacific time". The relevant email text: "So far, the concerns I've heard (such as identifying whether a pointer is within a certain block of memory) can be bypassed by secret channels between the standard library and the compiler."

If this is similar territory, the author might want to talk to Jens and possibly Arthur. If it's not, I don't have an issue with making it linear in the number of elements *in constexpr contexts only*. Might slow down compilation for constexpr code a bit though.

2. We do not have any data on the performance difference between a constexpr-enabled/disabled hive at this point in time, hopefully there should be none (see History section for explanation). However what we do have is data on the performance difference between one reusing the erased element memory space to store free list data, and one which allocates additional memory and does not reuse that memory space. This is from the change to v4.5 of colony (previous name for hive) and was detailed on my blog here: <https://plflib.org/blog.htm#colony45results> (note that v4.5 was an early unoptimized version of the approach compared to later).

The secondary advantage of reusing the erased element space is lowered memory requirement. Not reusing the erased element memory space would result in, for example, a 16-bit type in hive using 40 bits per element (ignoring block metadata), instead of 24 bits, in the reference implementation (or 17 bits in an implementation I had planned). The effect of this on memory usage and performance via cache waste should be obvious to everyone by this point.

3. The undefined behaviour (UB) within the reference implementation mentioned in the paper is necessary to support overaligned types - required by standard library containers - while retaining the ability to reuse the erased element memory space, when using `reinterpret_cast` for the type-punning. That's because there is no mechanism to supply custom alignment to an allocator in the C++ allocator model. Somebody else suggested that change, and till december '25, nobody had informed me it was UB. It would've been a nice thing to know earlier. Hana was made aware of the usage prior to the NB discussion and provided no feedback.

I've been informed post-kona that this same mechanism is used elsewhere in STL implementations for the same reason (overalignment + allocators + type punning), such as <https://cplusplus.github.io/fundamentals-ts/v3.html#memory.resource.adaptor>
So presumably it's not considered UB that can generally cause actual issues.

4. Given that it is UB, presumably the only way to reuse the erased element memory space in hive without UB is C++23 unions. However it should be noted that the paper's union-based

implementation at this point also contains UB related to unions for optimisation purposes, at the time of writing. Noted at line 2200 in <hive> (<https://github.com/NylteJ/STL/blob/a64775cf72ea52095306425d84c57e3979ea5ac9/stl/inc/hive#L2364>, may be moved/removed by the time you read this as it is a WIP implementation, if so search for 'this is UB'). It is labelled as 'technically' UB, which I take to mean UB which will never actually create issues ie. exactly the same as the reinterpret_cast UB in my code. The exploration of these sorts of issues is why we need an implementation first. So both implementations presently have non-problematic UB, but only one can potentially work in constexpr.

5. There is an insistence in the paper that we make the container constexpr before publication, however, we managed to do it post-publication for all the containers which've been around for 32 years. Therefore there is in fact no rush, and while it might be ideal to get it in C++26, it is in my view better to have a fully-functional constexpr implementation and thorough understanding, and third-party testing of the implementation, rather than rushing in. Runtime performance and latency takes priority over constexpr-ness, for most SG14 devs. It might be preferable to wait until other major std libs have created their own implementations, just to give leeway for them to explore alternative, potentially-better routes which could be constexpr-averse, before eventually adapting their strategies to be constexpr-friendly - either via adapting their methods, or via in-code consteval branches.
6. In case there's anyone doubting the substantiality of the latter, consider that there's already a performance optimization for hive which's not possible in constexpr mode: branch-free iteration on memory-reduced implementations. In P0447->Appendix I->Additional info for supporting small types, I describe a method of supporting byte-sized types without artificially widening the type storage (this's actually implemented in the MS implementation, which's good). Here the jump-counting data is stored in the erased element memory space (EEMS) with the erasures indicated in a bitset. This approach can be extended to larger types, and also to larger block capacities via [this method](#).

To make ++ iteration branch-free here, one multiplies the bitset bit at the current index by the value of the element/EEMS (memcpy'd to a temporary of the same numeric type as the jump-counting info) and adds that to the current index. Thereby if the bit is 1 (EEMS contains jump-counting info), a skip occurs, if 0 (no erasure) it doesn't - which works regardless of whether or not the element memory location contains an element or jump-counting data. Memepty (which doesn't work in constexpr as yet) is the only method to do this which avoids UB (the numeric representation of a non-trivially-copyable object is implementation-defined not undefined, but we multiply it by zero so it doesn't matter), as bit_cast doesn't work with non-trivially-copyable types. The pessimisation implied here is non-problematic - we can use 'if consteval' to switch to the branching method during constexpr processing, and use the branch-free method otherwise. But it serves as an example of how non-constexpr exploration is worthwhile in terms of performance.

II. History correction

If anyone has as little interest in the history of constexpr in hive, as I do, they can skip this. To be fair, some of the incorrectness of the history section in the paper is based on the incorrect understanding I

had of constexpr in early development of the paper, but some is misquotation. The history of hive, like the history of programming in general, is a history of making mistakes and either correcting oneself or being corrected by others. I have no difficulty with that.

1. I never built a constexpr version of hive. I believed I had at the time, but did not understand that reinterpret_cast wasn't allowed at compile time, at the time of writing D0447R13 (march 2021). Hence I basically took the existing hive implementation and tacked constexpr onto all the functions. The result, due to the incompleteness or incorrectness of constexpr support in compilers at the time, was that when compiling runtime code, they bloated the codegen compared to the non-constexpr-tagged container version, and had a 1-2% performance loss, variation dependent on the benchmark in question. I assumed this was due to the compiler inappropriately precalculating a lot of values instead of evaluating them at runtime, which may or may not have been the case.

Later versions of the same compilers did not produce this bloat/slow response, so I noted that in R20 and removed the info in subsequent updates, as I realised this behaviour was not an intended function of constexpr. The text in P0447R20, which Hana has omitted in her paper, reads: 'UPDATE May 2022: Re-testing these assumptions, later compiler versions appear to not suffer from the issues around codegen and speed (although codegen is still very much different for versions of the container with constexpr specified for all functions). There are still however issues with compilation and I think a staggered approach to rolling out constexpr containers is still the right one, so that any potential "gotcha"s can be addressed before ABIs in the various STL flavours essentially become frozen.'

Hana has written of R20: "The author says adding of constexpr can be done later before ABI freezes." - which is not what I said - my recommendation was for what essentially happened - a staggered rollout of constexpr containers, so the ABI of the constexpr versions didn't have to change due to subsequent discovered issues with constexpr containers. What I experienced earlier seemed to me a strong indication of how constexpr container support still wasn't ready for production *at that time*. Within a year the compilation issues I had experienced also disappeared in newer versions of the compilers, just in time for C++23.

2. At the point in time when the FAQ relating to constexpr usage was last updated (pre-C++23), vector and array were the only constexpr containers, and there was still no mechanism available at compile time to re-use the erased element memory space for other data, in order to reduce memory use and improve performance. I do not have time or energy to keep track of all the changes in the standard. Nobody informed me of the change to unions in C++23 making them constexpr-friendlier. Prior to that there was no practical way to reuse the erased element memory space as noted above, in constexpr, making it a non-starter without creating 2 separate versions of hive implementations - one for realtime with the reuse, one for compile time without - ie. a double burden on implementors.

III. Implementation comments

1. The build instructions don't work for the supplied branch, and there is no single standalone header to work with. I personally consider a standalone header (with whatever sub-header dependencies

included) to be a necessity for proper group audit. From the point of view of members being able to independently test given scenarios. Or failing that, working instructions for build, but as few barriers to participation as is possible. There may be simple steps to getting it working that are obvious to those involved, but they're not present. I don't consider being limited to specific version of MSVC a significant barrier for testing. At the moment, I can't build something with it, so I have no idea whether it functions and cannot give a thumbs up for constexpr hive on that basis.

2. Although I'm uncertain on the legitimacy of the claim in this code comment - <https://github.com/NylteJ/STL/blob/a64775cf72ea52095306425d84c57e3979ea5ac9/stl/inc/hive#L4246> - it sounds like it's saying the requirement in `block_capacity_hard_limits()` to not return a `.max` larger than `allocator_traits::max_size()` could be problematic for custom default-constructible allocators. If this is believed to be the case, it should be made a LWG issue as soon as possible so that it can be discussed.

The function itself is of course necessary for allowing devs to check a given implementation's architectural limits primarily, not the allocator's.

3. The current benchmarks in the repo are OK for testing internal functions for performance changes. If the authors want to do things like test the effects of differing block capacities on performance, or compare against other containers, or compare different internal strategies for overall performance, a real-world emulation scenario is called for eg. inserting, erasing and iterating to a single hive, switching between the 3 over time, and timing that. This is what happens in my unordered modification and 'referencer' benches (https://plflib.org/benchmarks_haswell_gcc.htm).

The reason for this is that a larger block capacity will almost always give stronger insertion and iteration performance when these are tested in isolation (up to a limit), due to the lower numbers of block transitions and, potentially, increased cache locality. However, these results don't take into account the vast amounts of wasted memory and cache waste that can result from a larger block + many erasures, which affects both insertion and iteration performance. With a smaller block capacity the statistical odds of a block becoming empty and removed from the iterative sequence is far greater, improving iteration performance by removing long erased-element skips (and insertion performance, as, all things being equal, inserting to the back typically takes less code). Hence you want benchmarks which do all 3 over long periods of time, not in isolation.

IV. Procedures comments

1. It would be best if people wishing to make changes to a paper were encouraged to talk to paper authors directly, initially, and early, to avoid misunderstandings/misgivings. Most people are rational and will admit an error in judgement, if it is an error, when sufficient evidence is given. And while an adversarial position can sometimes benefit the committee, generally it is to be avoided. I discovered randomly, while searching for a particular conversation about hive during kona 25, that Hana had brought up the topic (constexpr hive) in mattermost circa Feb '25, and was told by a LWG member that it would be best to wait for real `std::lib` implementations first, and there was no rush. But 8 months passed and the NB was defended without a functional `std::lib` implementation

mentioned. It would be better that people write their own paper and/or have discussions where there is time, rather than skipping to NB's, if they want to get something into a paper.

2. I think NB's reflecting major changes to papers need to be voted on in the context of pursuing the idea further, not as confirmation/deconfirmation of the idea, circumstances-dependent. We had several instances this NB session with large issues not being given adequate time, and paper authors not getting a chance to adequately defend their position. Regardless of rightness of outcome in those situations, this tends to lead to a distrust and distaste for committee proceedings, in the community in general. A vote on an NB for a large change should be a vote that the committee wants a either a paper, subsequent discussion, an implementation or some combination of those. NB comments are numerous and time therefore limited, and I think this constitutes a rational way forward in future. Anyway - stepping out of the way for now -
Matt